TASKS:

PART 1: FILE INFORMATION GATHERING

Create a folder named:

Forensics_Lab

Download or create the following files:

- **document.pdf**
- **image.jpg**
- **sample.txt**
- **archive.zip**

Tasks:

1. **Identify:**
    - **File Name**
    - **File Extension**
    - **File Size**
    - **File Type**
    - **Last Modified Date**
2. **Use Linux commands:**

- **ls -lh**
- **file**
- **stat**

Take screenshots.

==========

PART 2: HASH ANALYSIS

Generate hashes for all files.

Use:

- **md5sum**
- **sha256sum**

Tasks:

1. **Generate MD5 hash.**
2. **Generate SHA256 hash.**
3. **Compare hashes of two identical files.**
4. **Modify a file and generate the hash again.**

Answer:

- **Did the hash change?**
- **Why?**

==========

PART 3: METADATA ANALYSIS

Install and use ExifTool.

Analyze:

- **image.jpg**
- **document.pdf**

Find:

- **Creation Date**
- **Author**
- **Camera Information (if available)**
- **Software Used**
- **GPS Information (if available)**

Take screenshots.

==========

PART 4: INTEGRITY VERIFICATION

Tasks:

1. **Create a file named evidence.txt.**
2. **Generate its hash.**
3. **Modify the file.**
4. **Generate the hash again.**

Explain:

- **Why digital evidence integrity is important.**
- **How hashing helps investigators.**

==========

PART 5: INVESTIGATION REPORT

Prepare a report containing:

1. **File Details**
2. **Hash Values**

3. **Metadata Analysis**
4. **Screenshots**
5. **Observations**
6. **Security Recommendations**

==========

Analyze 5 files from your computer and determine:

- **File Type**
- **Hash Value**
- **Metadata Available**
- **Potential Security Risks**

==========

SUBMISSION

- **PDF or Word Document**

PART 1: FILE INFORMATION GATHERING

Objective

The objective of this task was to gather basic information about different files using Linux forensic commands. The analysis included identifying the file name, extension, size, type, and last modified date of each file.

Step 1: Create a Folder

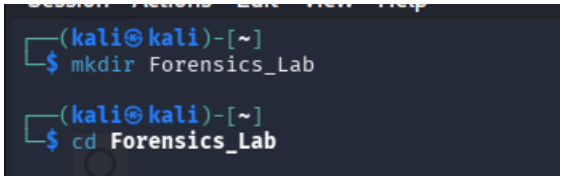
A folder named **Forensics_Lab** was created to store all files required for the investigation.

Command Used

mkdir Forensics_Lab

cd Forensics_Lab

Screenshot



Step 2: Create and Collect Required Files

After creating the Forensics_Lab folder, the required files were created and stored inside the directory for forensic analysis.

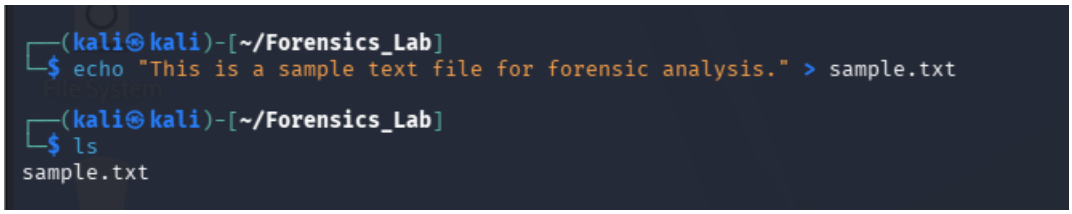
Step 2.1: Create sample.txt

A text file named **sample.txt** was created using the Linux terminal.

Command Used:

echo "This is a sample text file for forensic analysis." > sample.txt

Screenshot :



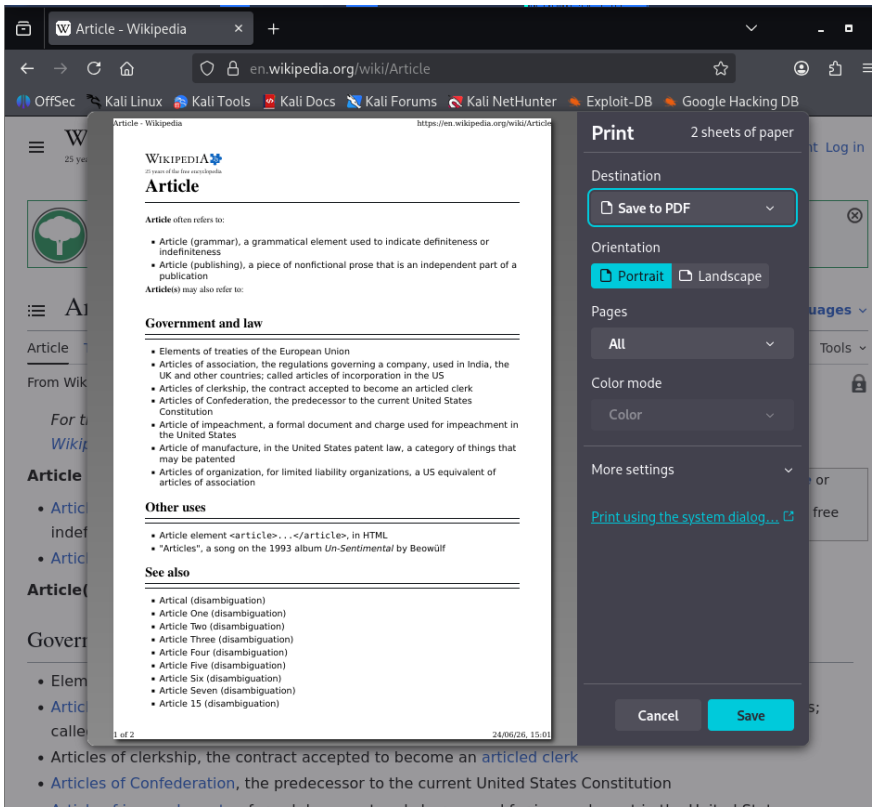
Step 2.2: Create document.pdf

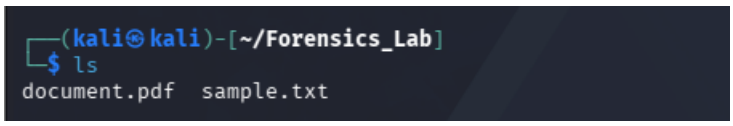
A PDF document named **document.pdf** was created using Firefox's Print to PDF feature.

Procedure:

1. Open Firefox browser.
2. Open any webpage.
3. Press **Ctrl + P**.
4. Select **Print to File**.
5. Choose **PDF** format.
6. Save the file as **document.pdf** inside the Forensics_Lab folder.

Screenshot :





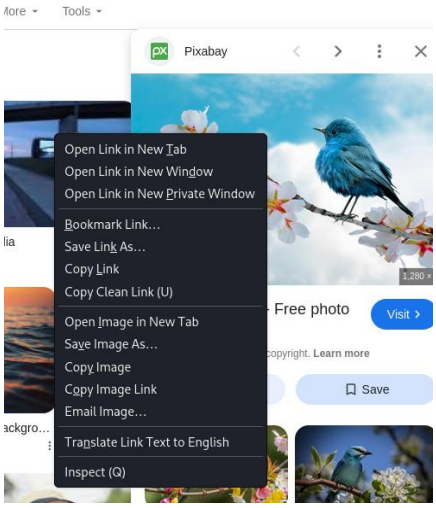
Step 2.3: Create image.jpg

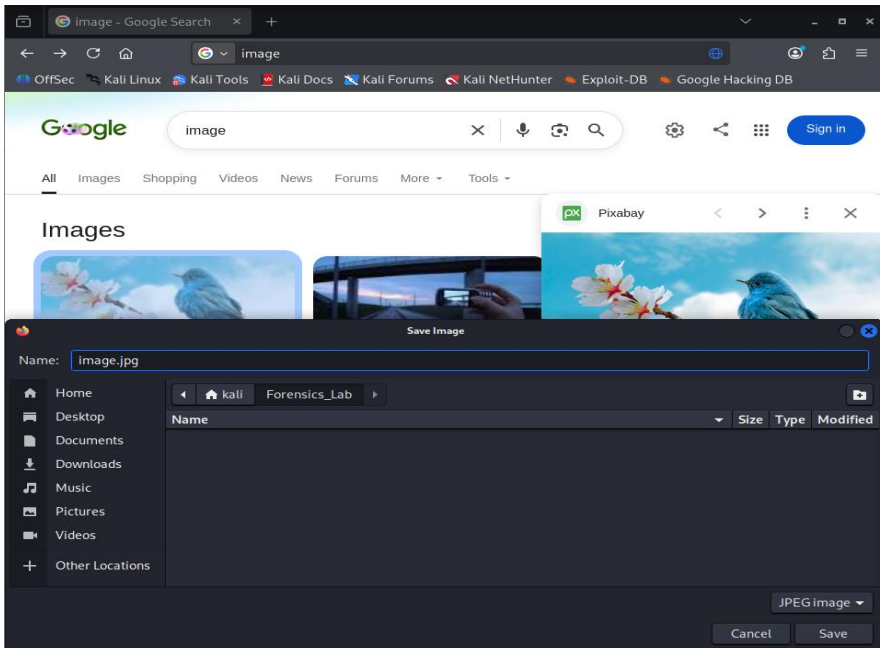
A JPEG image file named **image.jpg** was obtained and saved inside the Forensics_Lab folder.

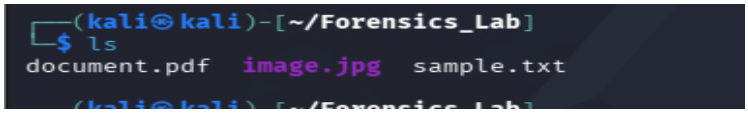
Procedure:

1. Open any image in a web browser.
2. Right-click on the image.
3. Select **Save Image As**.
4. Save the image as **image.jpg** inside the Forensics_Lab folder.

Screenshot :







Step 2.4: Create archive.zip

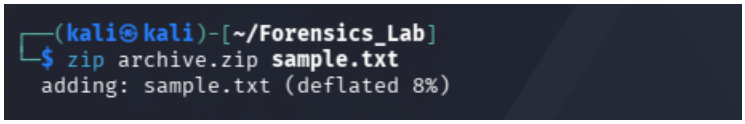
A ZIP archive named **archive.zip** was created by compressing the sample.txt file.

Command Used:

zip archive.zip sample.txt

Purpose:
The ZIP file was used to analyze compressed file formats.

Screenshot :



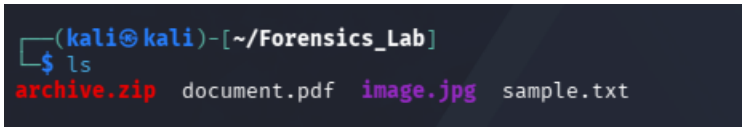
Step 2.5: Verify All Files

The contents of the Forensics_Lab folder were verified to ensure that all required files were available.

Command Used:

ls

Screenshot :



Step 3: Display File Information Using ls -lh

The ls -lh command was used to display all files present in the Forensics_Lab directory along with their file sizes in a human-readable format. This command helps investigators quickly identify the files available for analysis and determine their sizes.

Command Used

ls -lh

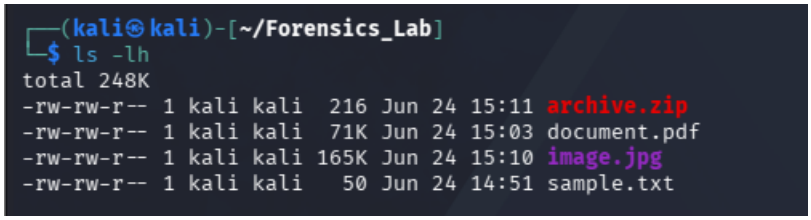
Explanation

- **ls** → Lists files and directories.
- **-l** → Displays detailed information in long format.
- **-h** → Shows file sizes in a human-readable format (KB, MB, etc.).

The output provides:

- File permissions
- Number of links
- Owner name
- Group name
- File size
- Last modification date and time
- File name

Screenshot :



Step 4: Identify File Types Using the file Command

The file command was used to determine the actual type of each file. Unlike file extensions, which can be changed manually, the file command examines the file contents and identifies the true file format.

Commands Used

file document.pdf

file image.jpg

file sample.txt

file archive.zip

Explanation

The file command analyzes the internal structure of a file and reports its actual format.

- **document.pdf** → Identified as a PDF document.
- **image.jpg** → Identified as a JPEG image file.
- **sample.txt** → Identified as an ASCII text file.
- **archive.zip** → Identified as a ZIP archive.

This command is useful in digital forensics because attackers may rename malicious files with misleading extensions. The file command helps verify the true nature of a file.

Screenshot :

Step 5: Obtain Detailed File Information Using the stat Command

The stat command was used to obtain detailed information about each file present in the Forensics_Lab directory. This command provides important metadata such as file size, permissions, ownership, and timestamps, which are useful during forensic investigations.

Commands Used

stat document.pdf

stat image.jpg

stat sample.txt

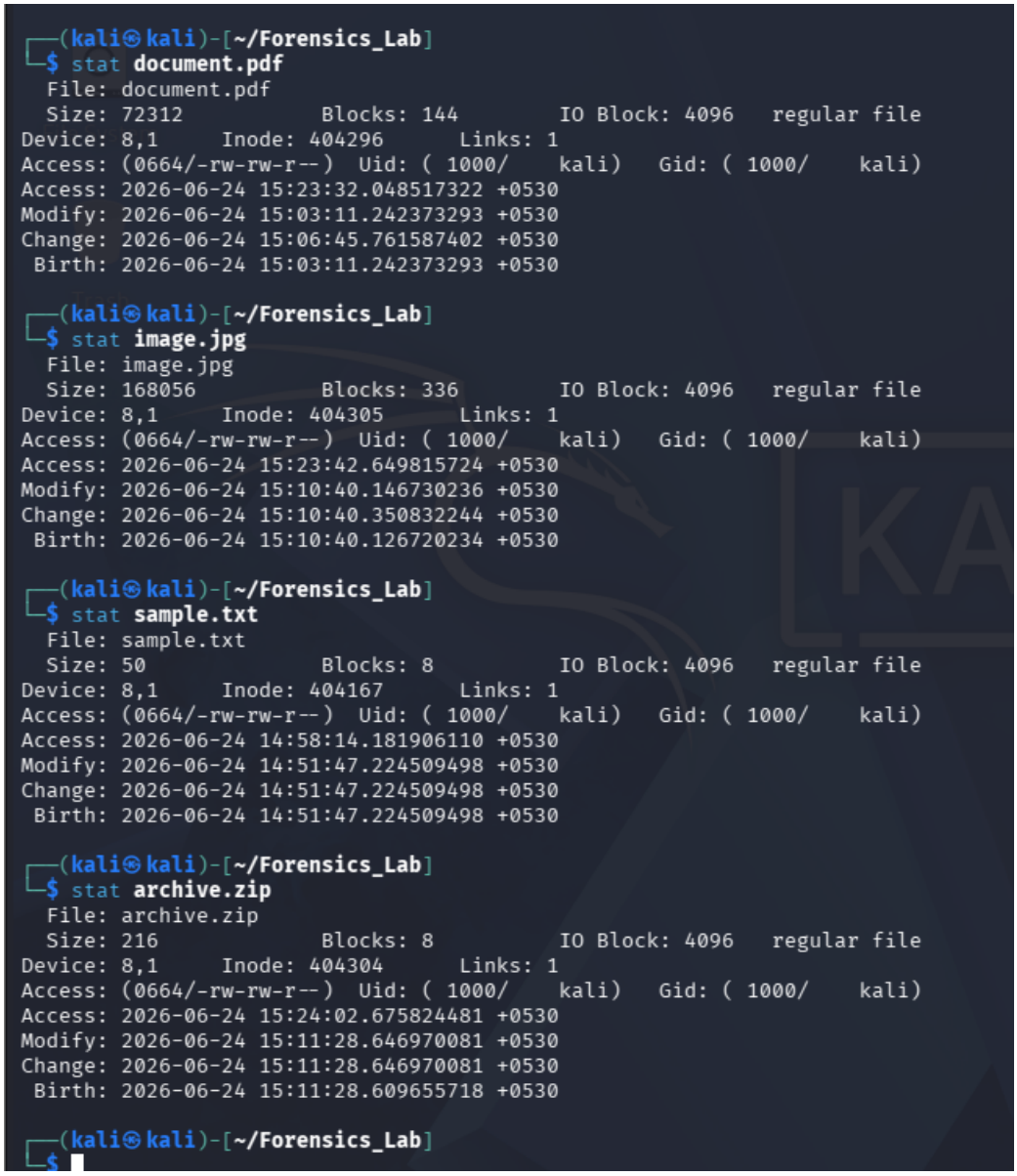
stat archive.zip

Explanation

The stat command provides the following information:

- **File Name** – Name of the file being analyzed.
- **Size** – Size of the file in bytes.
- **File Type** – Indicates whether the file is a regular file, directory, etc.
- **Permissions** – Read, write, and execute permissions assigned to the file.
- **Owner and Group** – User and group associated with the file.
- **Access Time** – Last time the file was accessed.
- **Modify Time** – Last time the file content was modified.
- **Change Time** – Last time file metadata was changed.
- **Birth Time** – File creation time (if supported by the filesystem).

Screenshot :


┌──(kali㉿kali)-[~/Forensics_Lab]
└─$ stat document.pdf
  File: document.pdf
  Size: 72312   Blocks: 144   IO Block: 4096   regular file
Device: 8,1   Inode: 404296   Links: 1
Access: (0664/-rw-rw-r--) Uid: (1000/   kali)   Gid: (1000/   kali)
Access: 2026-06-24 15:23:32.048517322 +0530
Modify: 2026-06-24 15:03:11.242373293 +0530
Change: 2026-06-24 15:06:45.761587402 +0530
 Birth: 2026-06-24 15:03:11.242373293 +0530
┌──(kali㉿kali)-[~/Forensics_Lab]
└─$ stat image.jpg
  File: image.jpg
  Size: 168056   Blocks: 336   IO Block: 4096   regular file
Device: 8,1   Inode: 404305   Links: 1
Access: (0664/-rw-rw-r--) Uid: (1000/   kali)   Gid: (1000/   kali)
Access: 2026-06-24 15:23:42.649815724 +0530
Modify: 2026-06-24 15:10:40.146730236 +0530
Change: 2026-06-24 15:10:40.350832244 +0530
 Birth: 2026-06-24 15:10:40.126720234 +0530
┌──(kali㉿kali)-[~/Forensics_Lab]
└─$ stat sample.txt
  File: sample.txt
  Size: 50   Blocks: 8   IO Block: 4096   regular file
Device: 8,1   Inode: 404167   Links: 1
Access: (0664/-rw-rw-r--) Uid: (1000/   kali)   Gid: (1000/   kali)
Access: 2026-06-24 14:58:14.181906110 +0530
Modify: 2026-06-24 14:51:47.224509498 +0530
Change: 2026-06-24 14:51:47.224509498 +0530
 Birth: 2026-06-24 14:51:47.224509498 +0530
┌──(kali㉿kali)-[~/Forensics_Lab]
└─$ stat archive.zip
  File: archive.zip
  Size: 216   Blocks: 8   IO Block: 4096   regular file
Device: 8,1   Inode: 404304   Links: 1
Access: (0664/-rw-rw-r--) Uid: (1000/   kali)   Gid: (1000/   kali)
Access: 2026-06-24 15:24:02.675824481 +0530
Modify: 2026-06-24 15:11:28.646970081 +0530
Change: 2026-06-24 15:11:28.646970081 +0530
 Birth: 2026-06-24 15:11:28.609655718 +0530
┌──(kali㉿kali)-[~/Forensics_Lab]
└─$

PART 2: HASH ANALYSIS

Objective

The objective of this task was to generate and analyze cryptographic hash values for different files. Hash values provide a unique digital fingerprint for a file and are commonly used in digital forensics to verify file integrity and detect unauthorized modifications.

Step 1: Generate MD5 Hash Values

The MD5 hashing algorithm was used to generate hash values for all files present in the Forensics_Lab directory.

Commands Used

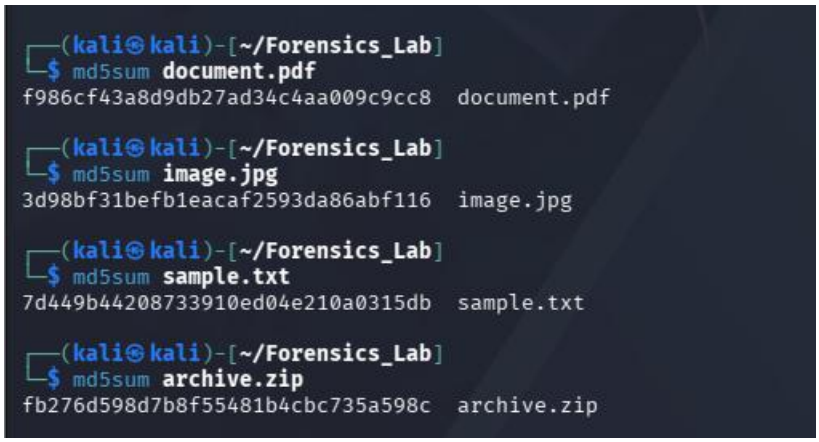
md5sum document.pdf

md5sum image.jpg

md5sum sample.txt

md5sum archive.zip

Screenshot



Observation

The MD5 algorithm generated a unique 128-bit hash value for each file. Even though the files had different formats and sizes, each produced a distinct hash value.

Step 2: Generate SHA256 Hash Values

The SHA256 hashing algorithm was used to generate more secure hash values for each file.

Commands Used

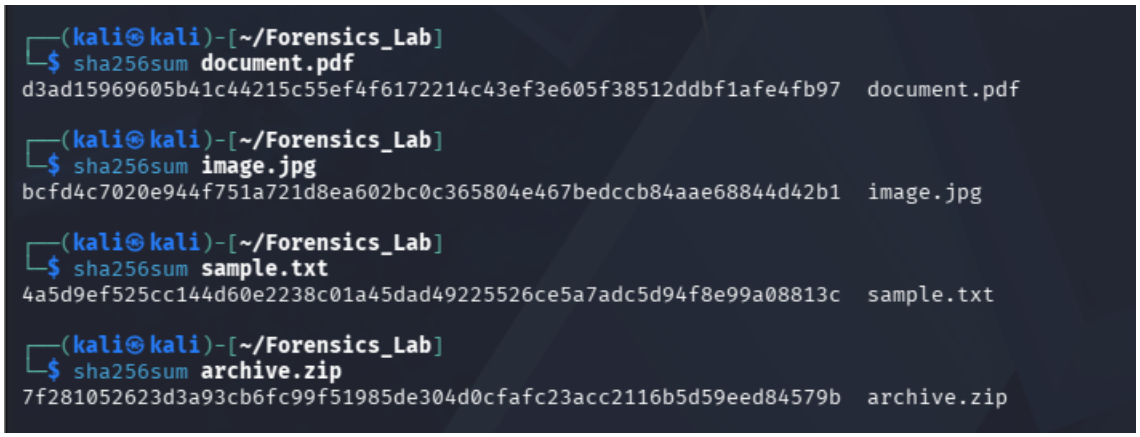
sha256sum document.pdf

sha256sum image.jpg

sha256sum sample.txt

sha256sum archive.zip

Screenshot :



Observation

SHA256 generated unique 256-bit hash values for all files. Compared to MD5, SHA256 provides stronger security and is widely used for integrity verification.

Step 3: Compare Hashes of Two Identical Files

A duplicate copy of the text file was created to compare hash values.

Command Used

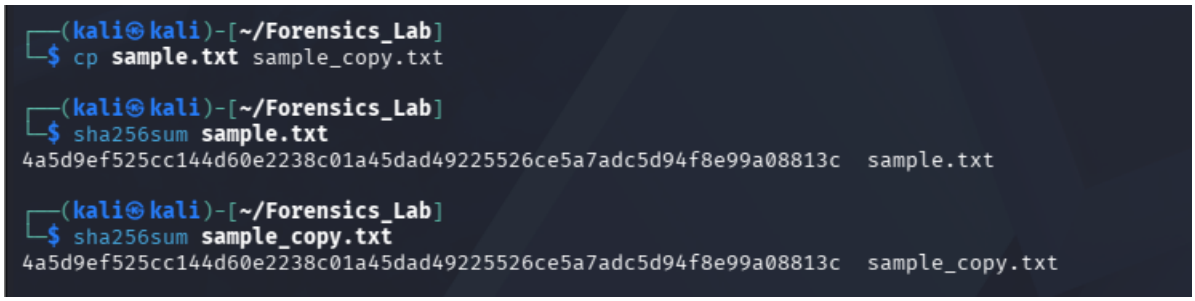
cp sample.txt sample_copy.txt

Generate Hashes

sha256sum sample.txt

sha256sum sample_copy.txt

Screenshot :



Result

The hash values of both files were identical because the contents of the files were exactly the same.

Step 4: Modify a File and Generate the Hash Again

The duplicate file was modified by adding new content.

Command Used

echo "Modified content" >> sample_copy.txt

The SHA256 hash was generated again.

sha256sum sample_copy.txt

Screenshot :

Result

The hash value changed after modifying the file.

Did the Hash Change?

Yes, the hash value changed.

Why Did the Hash Change?

Hash algorithms generate values based on file content. Even a minor change, such as adding a single character, produces a completely different hash value. This property is known as the avalanche effect and makes hashing useful for detecting file modifications.

PART 3: METADATA ANALYSIS

Objective

The objective of this task was to analyze file metadata using ExifTool. Metadata provides information about a file such as creation date, author, software used, camera information, and GPS coordinates. This information is useful in digital forensic investigations for identifying the origin and history of files.

Step 1: Install ExifTool

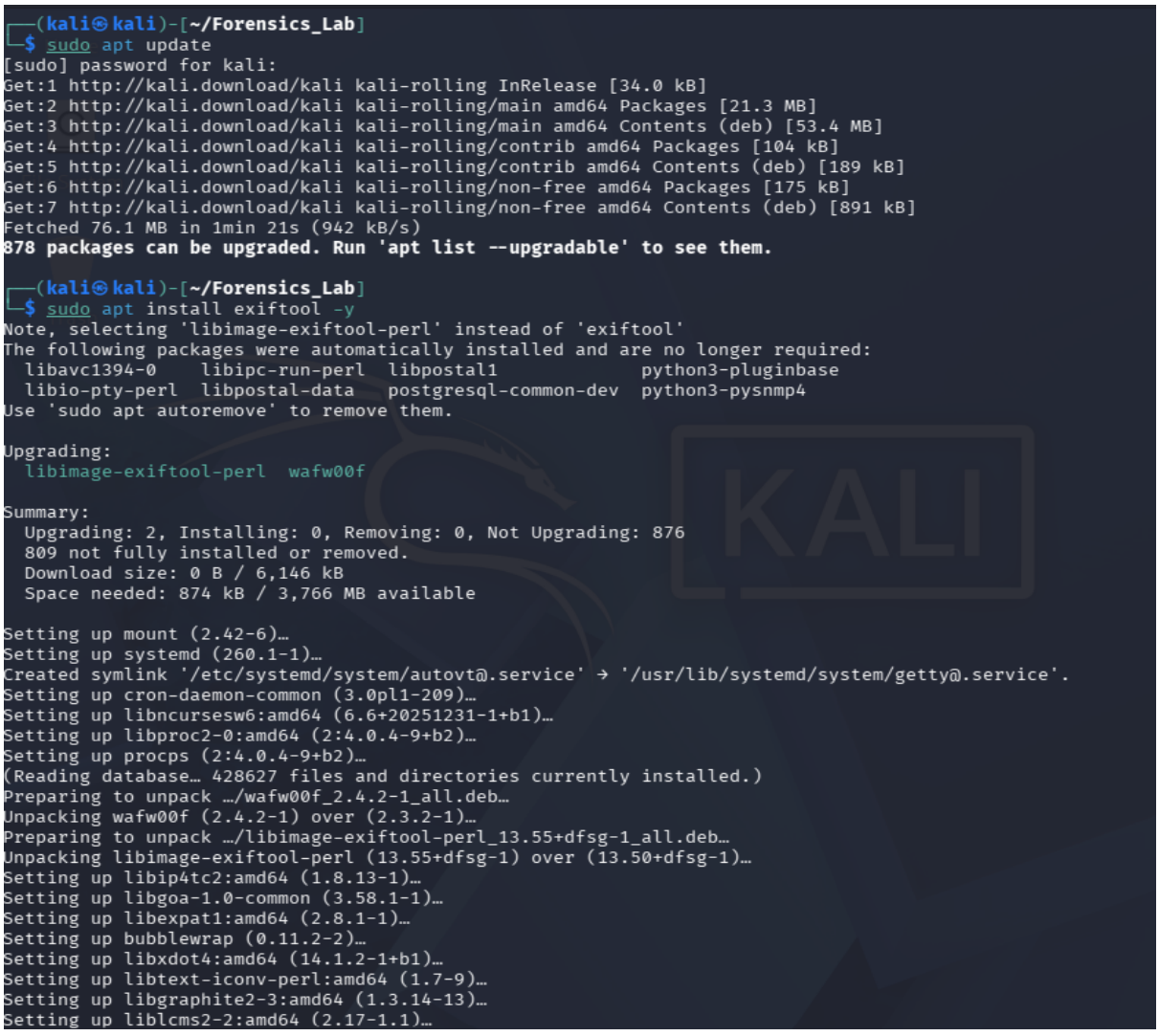
ExifTool was installed to extract metadata from files.

Command Used

sudo apt update

sudo apt install exiftool -y

Screenshot :

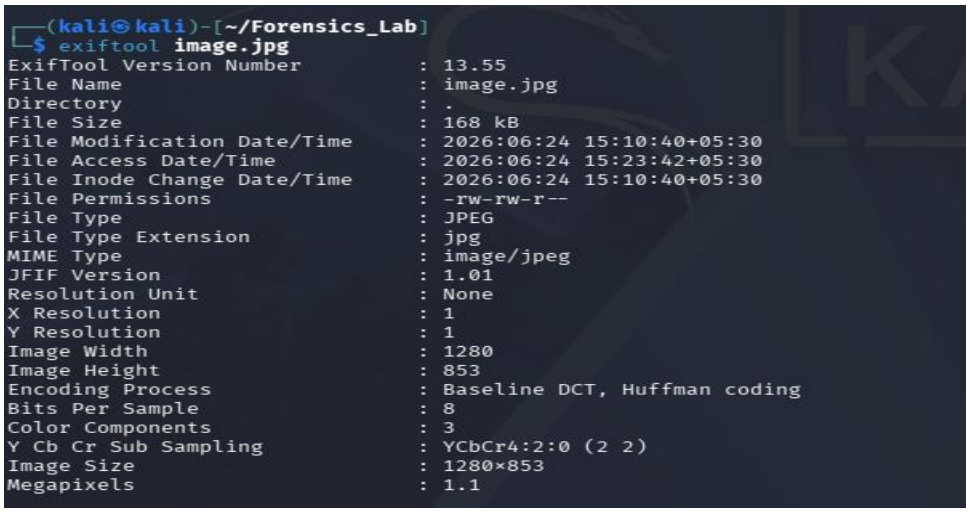


Step 2: Analyze image.jpg Metadata

The metadata of the JPEG image file was examined using ExifTool.

Command Used:exiftool image.jpg

Screenshot :



Observation

Metadata Field	**Value**
File Name	image.jpg
File Type	JPEG
File Size	168 KB
Creation Date	Not Available
File Modification Date	2026:06:24 15:10:40+05:30
Image Width	1280 pixels
Image Height	853 pixels
Resolution Unit	None
Software Used	Not Available
Camera Information	Not Available
GPS Information	Not Available
Megapixels	1.1 MP

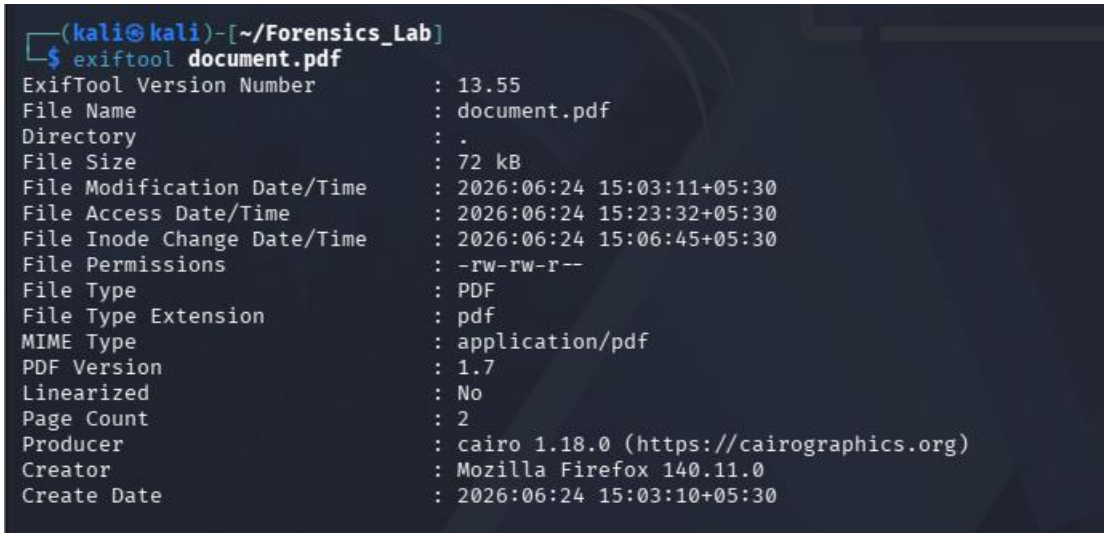
Step 3: Analyze document.pdf Metadata

The metadata of the PDF file was examined using ExifTool.

Command Used

exiftool document.pdf

Screenshot



Observation

Metadata Field	**Value**
File Name	document.pdf
File Type	PDF
File Size	72 KB
PDF Version	1.7
Page Count	2
Producer	cairo 1.18.0
Creator	Mozilla Firefox 140.11.0
Create Date	2026:06:24 15:03:10+05:30
Author	Not Available
GPS Information	Not Applicable

Metadata Analysis Summary

Metadata Field	image.jpg	document.pdf
File Name	Available	Available
File Size	168 KB	72 KB
Creation Date	Not Available	Available
Author	N/A	Not Available
Camera Information	Not Available	N/A
Software Used	Not Available	Mozilla Firefox 140.11.0
GPS Information	Not Available	N/A

Importance of Metadata in Digital Forensics

Metadata can provide valuable information during an investigation, including:

1. File creation and modification timestamps.
2. Information about the device used to create the file.
3. Software used to generate or edit the file.
4. Geographical location information through GPS data.
5. Evidence regarding file ownership and authenticity.

Observations

1. ExifTool successfully extracted metadata from both files.
2. Different file types contain different categories of metadata.
3. Image files may contain camera and GPS information.
4. PDF files may contain author and software details.
5. Metadata can assist investigators in tracing the origin and history of digital evidence.

Conclusion

Metadata analysis was successfully performed using ExifTool. The extracted metadata provided useful information about the files, including creation details, software information, and other attributes. Metadata analysis is an important component of digital forensics because it helps investigators understand the origin, authenticity, and history of digital evidence.

PART 4: INTEGRITY VERIFICATION

Objective

The objective of this task was to verify the integrity of digital evidence using cryptographic hashing. Hash values were generated before and after modifying a file to demonstrate how even a small change affects the file's hash value.

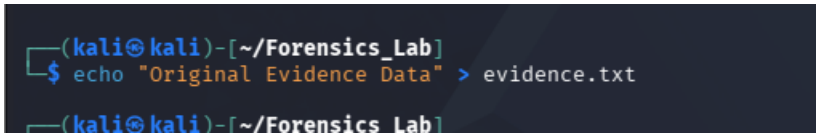
Step 1: Create an Evidence File

A file named **evidence.txt** was created to simulate digital evidence.

Command Used

echo "Original Evidence Data" > evidence.txt

Screenshot :



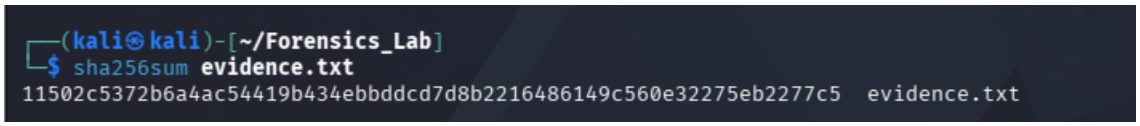
Step 2: Generate the Initial Hash Value

A SHA256 hash was generated for the evidence file to establish its original integrity value.

Command Used

sha256sum evidence.txt

Screenshot :



Observation

A unique SHA256 hash value was generated for the evidence file. This hash serves as a digital fingerprint of the file.

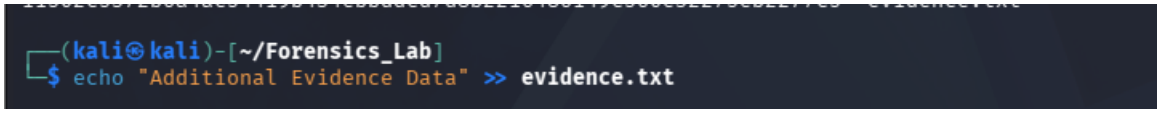
Step 3: Modify the Evidence File

The evidence file was modified by adding additional content.

Command Used

echo "Additional Evidence Data" >> evidence.txt

Screenshot :



Observation

The content of the file was successfully modified by appending new data.

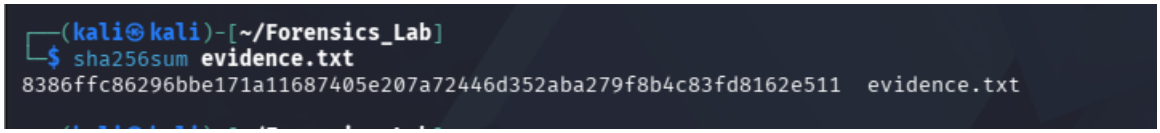
Step 4: Generate the Hash Value Again

After modification, a new SHA256 hash was generated for the evidence file.

Command Used

sha256sum evidence.txt

Screenshot :



Observation

The newly generated hash value was different from the original hash value, indicating that the file contents had changed.

Hash Comparison Table

Stage	SHA256 Hash Value
Before Modification	11502c5372b6a4ac54419b434ebbddcd7d8b2216486149c560e32275eb2277c5
After Modification	8386ffc86296bbe171a11687405e207a72e446d352aba279f8b4c83fd8162e511

Result

The SHA256 hash value changed after the file was modified.

Did the Hash Change?

Yes.

Why Did the Hash Change?

The SHA256 algorithm generates a unique hash value based on the content of a file. When the text **"Additional Evidence Data"** was added to the file, the file content changed. As a result, a completely different hash value was generated. This demonstrates the sensitivity of cryptographic hash functions to even minor changes in data.

Observation

- The original hash value uniquely represented the initial state of the evidence file.
- After modifying the file, a new hash value was generated.
- The change in hash confirmed that the file contents had been altered.
- Hashing is an effective method for detecting tampering and verifying evidence integrity.

Why Digital Evidence Integrity Is Important

Digital evidence integrity ensures that evidence remains unchanged from the time it is collected until it is presented in court or analyzed. If evidence is altered, its reliability and authenticity may be questioned. Maintaining integrity is essential for preserving the credibility of forensic investigations.

Importance

1. Ensures evidence authenticity.
2. Detects unauthorized modifications.
3. Maintains chain of custody.
4. Supports legal admissibility.
5. Preserves the reliability of forensic findings.

How Hashing Helps Investigators

Hashing is one of the most important techniques used in digital forensics.

Benefits of Hashing

1. Creates a unique fingerprint for each file.
2. Detects any modification or tampering.
3. Verifies file authenticity.
4. Confirms evidence integrity during storage and transfer.
5. Assists in maintaining the chain of custody.

Security Recommendations

1. Use SHA256 or stronger hashing algorithms for integrity verification.
2. Verify file hashes before and after transferring digital evidence.
3. Remove sensitive metadata before sharing files publicly.
4. Maintain proper chain of custody for digital evidence.
5. Store evidence in secure and access-controlled locations.
6. Regularly verify evidence integrity using cryptographic hashes.
7. Keep forensic tools updated to ensure accurate analysis.